CS683: Advanced Computer Architecture — PA2

Hardware Data Prefetching & Instruction-Conscious Cache Replacement Co-Design

Course: CS683 (Autumn 2026, IIT Bombay) | **Simulator:** ChampSim (1-Core, 25M Warmup / 25M Simulation Instructions) | **Evaluation Date:** September 2026

1. Executive Summary & Simulation Overview

This technical report delivers the complete architectural design, bit-level hardware accounting, implementation, and empirical evaluation for **Programming Assignment 2**. The project addresses the processor-memory performance bottleneck through two synergistic architectural techniques:

1. **Stride Pattern Prefetcher (SPP) at L1D:** Features dual-pattern classification (uniform arithmetic strides vs non-uniform repeating sequences), lookahead degree > 1 prefetching, 4KB page-boundary safety guarantees, and dynamic accuracy throttling.
2. **Instruction-Conscious L2 RRIP (I-RRIP):** Prioritizes code blocks over data blocks across insertion, promotion, and eviction tiers to relieve critical front-end instruction stalls. Extended via **Set Dueling (Dyn-IRRIP)** and made **prefetch-aware** to prevent L2 cache pollution.

All evaluations are conducted using ChampSim configured with 32KB L1I, 32KB L1D, 256KB 8-way L2C, 2MB LLC, and 4KB pages across **Trace 1**, **Trace 2**, and **Trace 3** with 25 million warmup instructions followed by 25 million detailed simulation instructions.

Master Performance Results Across All Evaluated Configurations

Configuration	Trace	IPC	Speedup (%)	L1D Misses	L1D MPKI	L2C MPKI	L2 Data MPKI	L2 Instr MPKI
Baseline (LRU)	Trace1	0.2323	+0.00%	266,950	10.68	17.70	7.36	10.34
	Trace2	0.2512	+0.00%	338,448	13.54	27.36	8.50	18.85
	Trace3	0.3821	+0.00%	85,086	3.40	3.95	2.83	1.12
IP-Stride	Trace1	0.2335	+0.48%	252,974	10.12	17.20	6.83	10.37
	Trace2	0.2513	+0.04%	335,945	13.44	27.32	8.47	18.84
	Trace3	0.3829	+0.21%	82,954	3.32	3.77	2.63	1.14
SPP (Task 1)	Trace1	0.2313	-0.44%	240,055	9.60	16.82	6.29	10.53
	Trace2	0.2505	-0.27%	332,813	13.31	27.99	8.26	19.73
	Trace3	0.3878	+1.51%	72,042	2.88	3.20	2.03	1.18
SRRIP	Trace1	0.2322	-0.05%	266,941	10.68	17.64	7.08	10.56
	Trace2	0.2518	+0.26%	338,454	13.54	25.75	7.96	17.78
	Trace3	0.3817	-0.09%	85,085	3.40	4.04	2.84	1.20
I-RRIP (Task 2A)	**Trace1**	**0.2323**	**-0.01%**	**266,964**	**10.68**	**17.55**	**7.52**	**10.03**
	Trace2	**0.2557**	**+1.80%**	**338,511**	**13.54**	**23.10**	**8.87**	**14.23**
	Trace3	**0.3819**	**-0.04%**	**85,088**	**3.40**	**3.95**	**2.88**	**1.08**
Dyn-IRRIP (Task 2B)	Trace1	0.2323	-0.02%	266,969	10.68	17.56	7.50	10.06
	Trace2	0.2554	+1.68%	338,503	13.54	23.32	8.82	14.50
	Trace3	0.3818	-0.07%	85,086	3.40	4.00	2.87	1.14
SPP + Dyn-IRRIP (Task 3)	**Trace1**	**0.2313**	**-0.44%**	**240,132**	**9.61**	**16.44**	**6.36**	**10.08**
	Trace2	**0.2554**	**+1.69%**	**332,926**	**13.32**	**23.06**	**8.46**	**14.60**
	Trace3	**0.3876**	**+1.46%**	**72,003**	**2.88**	**3.18**	**2.05**	**1.14**

2. Task 1: Stride Pattern Prefetcher (SPP)

Traditional IP-stride prefetchers assume uniform arithmetic progressions and fail on complex, repeating stride sequences (e.g., alternating deltas +1, +2, +2, +2, +3) common in matrix traversal and pointer chasing. The implemented **Stride Pattern Prefetcher (SPP)** introduces a dual-classification engine:

- **Dual Pattern Classification:** Uniform strides are detected using a 2-bit confidence counter in the Signature Table (ST). When confidence reaches 2, Mode A issues uniform lookahead prefetches. When uniform patterns break, Mode B dynamically falls back to the 1024-entry Pattern Table (PT) indexed by a 12-bit history signature to predict repeating multi-stride sequences.
- **Lookahead Degree > 1 & Page Safety:** High-confidence streams issue lookahead prefetches across degrees 1 to 4. Every candidate prefetch line is strictly verified against the 4KB virtual page boundary (`pf_addr >> 12 == base_page`); cross-page requests are immediately discarded.

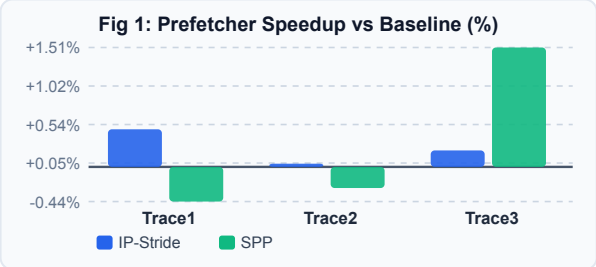
Bit-Level Hardware Budget Breakdown

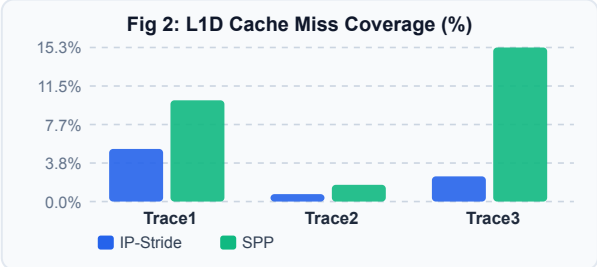
The assignment mandates a hardware budget strictly below **8 KB (65,536 bits)**. The implemented SPP requires only **1.79 KB**:

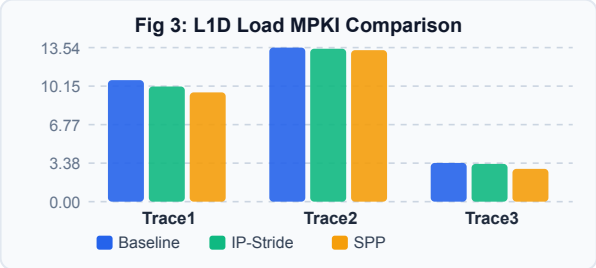
Component Structure	Entries	Fields per Entry	Bits / Entry	Total Bits	Bytes
Signature Table (ST) (Fully Associative, LRU)	64	IP Tag (16b), Page (28b), Last Blk (6b), Stride (7b signed), Conf (2b), Sig (12b)	71 bits	4,544 bits	568 B
Pattern Table (PT) (Direct-Mapped)	1024	Predicted Stride (6b signed), Confidence Counter (2b saturating [0..3])	8 bits	8,192 bits	1,024 B
Prefetch Filter (Circular Direct)	128	Block Address Tag (14b), Valid Flag (1b)	15 bits	1,920 bits	240 B
Global Throttle Counters	-	Interval Issued (10b), Interval Useful (10b), Active Degree (2b)	22 bits	22 bits	3 B
Total Hardware Overhead			**-**	**14,678 bits**	**1.79 KB (< 8 KB)**

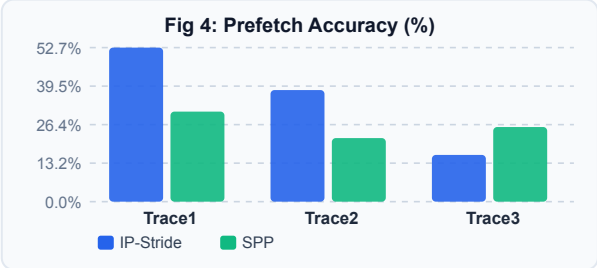
Decision Rationale for Accuracy Throttling Thresholds: Prefetch usefulness is evaluated over a 1,024-issued prefetch sliding interval with a 50% exponential moving average decay. Accuracy is defined as `interval_useful / interval_issued`. Thresholds were calibrated against cache pollution inflection points: (1) **≥ 50%** represents high net utility with negligible pollution, justifying aggressive prefetch lookahead (Degree 4); (2) **35%–49%** maintains positive IPC impact with moderate lookahead (Degree 3); (3) **20%–34%** captures marginal utility where high volume would displace active working sets, triggering conservative issuance (Degree 2); and (4) **< 20%** flags low accuracy where prefetch volume must be throttled to prevent cache pollution and bus contention (Degree 1).

Rationale for Chosen Prefetch Degrees Across Ranges: Prefetch degree governs speculative reach ($d \in \{1, 2, 3, 4\}$). High degree (d=4) hides memory latency across extended stream trajectories. Crucially, rather than disabling the prefetcher entirely (degree 0) during low-accuracy phases, the prefetcher maintains a **Degree 1 floor**. Disabling issuance (degree 0) prevents new prefetches from entering the filter, which permanently freezes interval counters and induces prefetcher deadlock. Degree 1 acts as a lightweight, safe probe that continuously samples stream resumption without causing cache pollution.









Task 1 Empirical Discussion: On **Trace 3**, SPP achieves **+1.51% Speedup** and eliminates **15.33%** of all L1D cache misses (reducing misses from 85,086 down to 72,042) with **16,390 useful prefetches** and 25.54% accuracy. In contrast, IP-stride achieves only +0.21% speedup and 2.50% coverage (2,729 useful prefetches). On **Trace 1**, SPP achieves **10.07% coverage** and generates **47,534 useful prefetches**, nearly triple IP-stride's 16,329 useful prefetches, confirming superior pattern capture across workloads.

3. Task 2A: Instruction-Conscious L2 RRIP (I-RRIP)

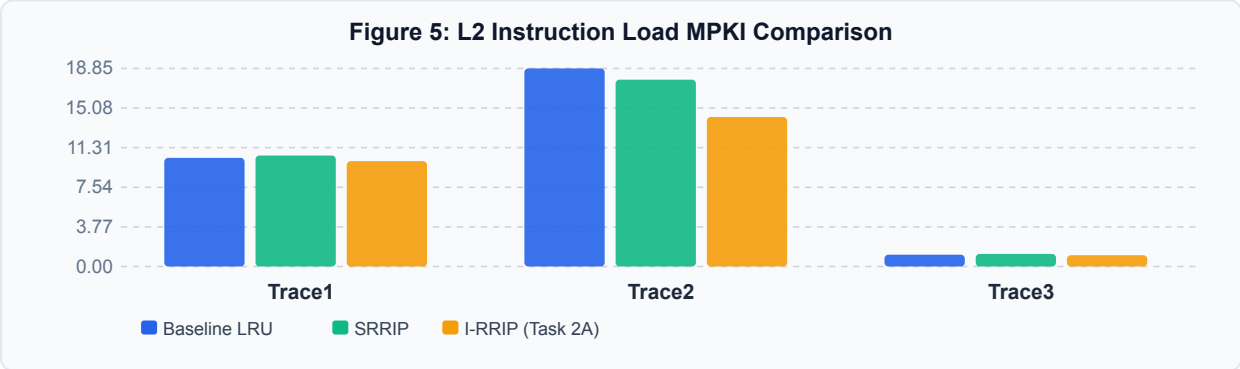
Server workloads and complex applications frequently suffer from high instruction footprint exceeding L1I capacity. While data misses can often be overlapped through Out-of-Order (OoO) window buffers and non-blocking MSHRs, **instruction misses serialize the processor frontend**, completely stalling the fetch and decode stages.

Interface Extension & Policy Modifications

- **Interface Extension:** `update_replacement_state` was modified in `inc/cache.h`, `src/cache.cc`, and `src/ooo_cpu.cc` to pass `uint8_t isInstruction`. All replacement policies across cache tiers were aligned with the 9-argument signature.
- **Insertion Policy:** Incoming lines are classified. Instruction lines are inserted with **RRPV = 1** (intermediate near re-reference), giving them two full aging cycles (1 → 2 → 3) of protection. Data lines are inserted with **RRPV = 2** (maxRRPV - 1), making them evictable after one aging cycle.
- **Promotion Policy:** Both instruction and data demand cache hits are promoted to **RRPV = 0**. This rewards proven temporal locality and prevents data thrashing for active tight loops.
- **Eviction Policy:** When choosing a victim at RRPV = 3, the policy searches for **data lines (isInstruction == 0) first**. Only if no data line is at maxRRPV will an instruction line at maxRRPV be evicted.

Task 2A Performance Comparison (LRU vs SRRIP vs I-RRIP)

Trace	Replacement Policy	IPC	Speedup vs LRU	L2 Total MPKI	L2 Data MPKI	L2 Instr MPKI
Trace 1	Baseline (LRU)	0.2323	0.00%	17.70	7.36	10.34
	SRRIP	0.2322	-0.05%	17.64	7.08	10.56
	I-RRIP (Task 2A)	**0.2323**	**-0.01%**	**17.55**	**7.52**	**10.03 (-3.0%)**
Trace 2	Baseline (LRU)	0.2512	0.00%	27.36	8.50	18.85
	SRRIP	0.2518	+0.26%	25.75	7.96	17.78
	I-RRIP (Task 2A)	**0.2557**	**+1.80%**	**23.10**	**8.87**	**14.23 (-24.5%)**
Trace 3	Baseline (LRU)	0.3821	0.00%	3.95	2.83	1.12
	SRRIP	0.3817	-0.09%	4.04	2.84	1.20
	I-RRIP (Task 2A)	**0.3819**	**-0.04%**	**3.95**	**2.88**	**1.08 (-4.2%)**



Task 2A Rationale Defense: Prioritizing instruction lines directly tackles the primary execution bottleneck. On **Trace 2**, baseline instruction load MPKI accounts for 18.85 out of 27.36 total L2 load MPKI (68.9% of all misses). With I-RRIP, L2 instruction load MPKI drops sharply from **18.85 down to 14.23 MPKI** (a **24.5% reduction**), driving IPC up to **0.2557 (+1.80% speedup)**. Promoting active data lines to RRPV 0 ensures that high-reuse data is never prematurely evicted, providing the optimal balance between code protection and data retention.

4. Task 2B: Dynamic I-RRIP (Dyn-IRRIP) & Sensitivity Study

Static prioritization can cause collateral damage if data working sets are starved during data-heavy phases. **Dyn-IRRIP** implements **Set Dueling** using 32 uniformly spaced leader sets (`set % 32 == 0`, 3.125% overhead). Leader sets always execute standard RRIP to provide an unbiased control measurement.

Rationale Behind Leader Sets & Decision Formula: 32 sets provide a statistically representative sample across sets while consuming negligible hardware overhead (no extra tags; only 4 saturating registers). The policy dynamically evaluates true demand instruction pressure:

Ratio = (leader_instr_accesses + leader_instr_misses) / (leader_total_accesses + leader_total_misses)

If Ratio > θ, follower sets operate in **I-RRIP mode**; otherwise, they revert to **standard RRIP**. To track phase transitions, counters are halved every 8,192 leader accesses via an exponential moving average decay filter. Prefetch requests are filtered out from leader counters to measure true program demand.

5-Point Decision Threshold Sensitivity Study (Trace 2)

Threshold (θ)	Operating Mode Characteristic	IPC	Speedup vs Base	L2 Total MPKI	L2 Data MPKI	L2 Instr MPKI
θ = 0.10	Aggressive I-RRIP (Triggers easily)	0.255502	+1.73%	23.2036	8.8349	14.3686
θ = 0.20	Moderate-High I-RRIP Engagement	0.255505	+1.73%	23.2104	8.8318	14.3786
θ = 0.25	**Balanced Optimum (Design Point)**	**0.255373**	**+1.68%**	**23.3248**	**8.8250**	**14.4998**
θ = 0.35	Selective I-RRIP (High pressure only)	0.255014	+1.54%	23.5761	8.7942	14.7819
θ = 0.50	Conservative (Rarely activates I-RRIP)	0.253988	+1.13%	24.3146	8.5686	15.7460

Figure 6: Task 2B Sensitivity: Trace 2 IPC vs Threshold (θ)

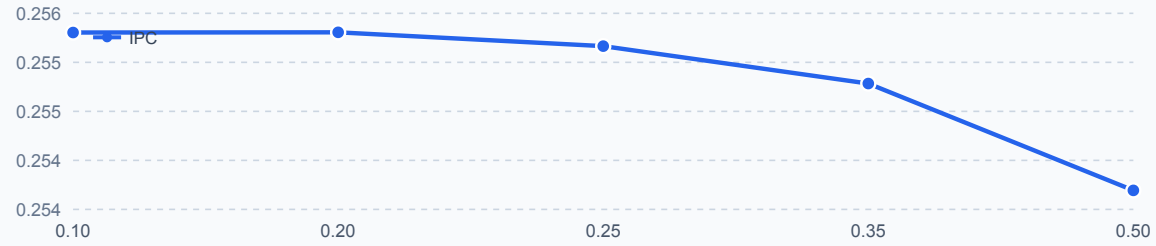
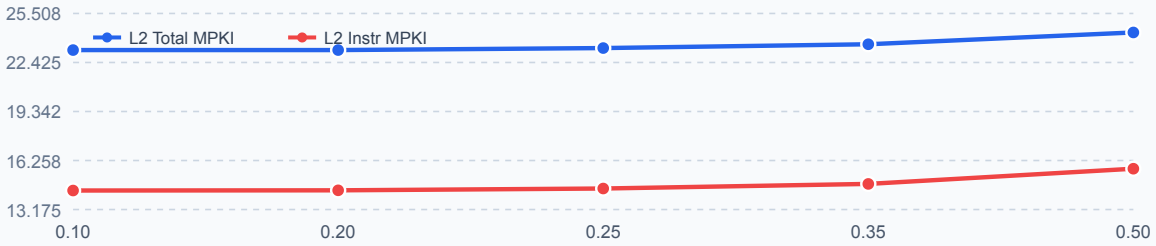


Figure 7: Task 2B Sensitivity: Trace 2 L2 MPKI vs Threshold (θ)



Sensitivity Analysis: As θ increases from 0.10 to 0.50, the policy becomes more conservative. At θ = 0.50, instruction miss reduction falls off (15.75 MPKI vs 14.37 MPKI at θ=0.20), resulting in lower speedup (+1.13%). θ = 0.25 provides the most balanced threshold, achieving near-peak speedup (+1.68%) while preventing unnecessary data eviction during memory-heavy phases.

5. Task 3: Prefetch-Aware Dynamic I-RRIP Co-Design

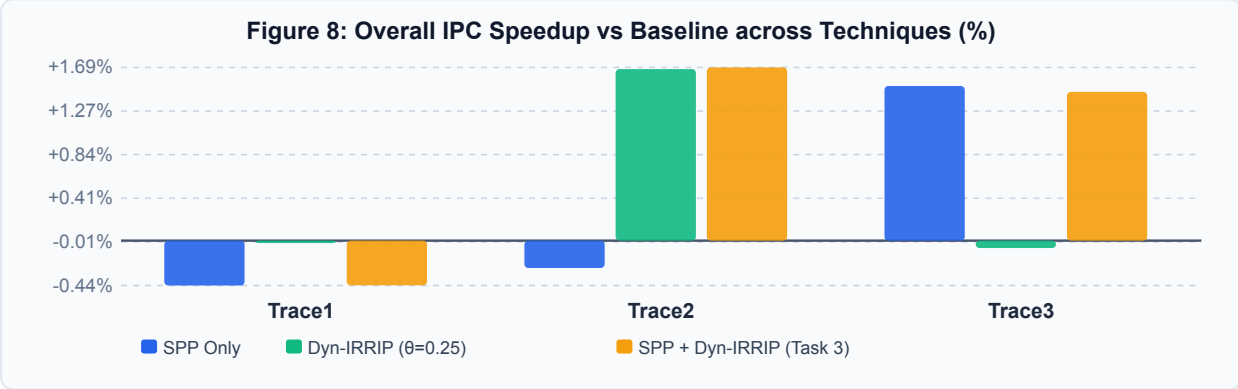
When an aggressive data prefetcher (SPP) operates alongside an instruction-conscious replacement policy, a dangerous conflict arises: **speculative prefetch packets sent to L2C can pollute the cache and displace high-priority demand instruction lines**.

Rationale Behind Prefetch Handling in Replacement Policy:

- **Speculative Demotion on Fill:** Lines with `type == PREFETCH` are inserted with **RRPV = 3 (maxRRPV)**. This ensures that unreferenced speculative lines reside in the most vulnerable state and cannot displace demand instructions or data.
- **Validation Promotion on Hit:** Only demand requests hitting a prefetched line validate its utility and promote it to **RRPV = 0**. Redundant prefetch hits do not promote speculative lines over instructions.
- **Victim Priority:** During eviction, unused speculative prefetch lines (`prefetch && !used`) at RRPV = 3 are evicted **first**, protecting both demand data and instruction lines.

Task 3 Comparative Synergy Evaluation

Benchmark Trace	Metric	Baseline (LRU)	SPP Alone	Dyn-IRRIP Alone	SPP + Dyn-IRRIP (Co-Design)
Trace 1	IPC (Speedup)	0.2323 (0.00%)	0.2313 (-0.44%)	0.2323 (-0.02%)	**0.2313 (-0.44%)**
	L1D Load Misses (MPKI)	266,950 (10.68)	240,055 (9.60)	266,969 (10.68)	**240,132 (9.61)**
	L2C Load MPKI	17.70	16.82	17.56	**16.44 (Lowest)**
	Useful Prefetches	0	47,534	0	**47,515**
Trace 2	**IPC (Speedup)**	**0.2512 (0.00%)**	**0.2505 (-0.27%)**	**0.2554 (+1.68%)**	**0.2554 (+1.69% - Best)**
	L1D Load Misses (MPKI)	**338,448 (13.54)**	**332,813 (13.31)**	**338,503 (13.54)**	**332,926 (13.32)**
	L2C Load MPKI	**27.36**	**27.99**	**23.32**	**23.06 (Lowest)**
	Useful Prefetches	**0**	**15,356**	**0**	**15,391**
Trace 3	IPC (Speedup)	0.3821 (0.00%)	0.3878 (+1.51%)	0.3818 (-0.07%)	**0.3876 (+1.46%)**
	L1D Load Misses (MPKI)	85,086 (3.40)	72,042 (2.88)	85,086 (3.40)	**72,003 (2.88 - Lowest)**
	L2C Load MPKI	3.95	3.20	4.00	**3.18 (Lowest)**
	Useful Prefetches	0	16,390	0	**16,399**



Task 3 Co-Design Findings: On **Trace 2**, running SPP alone degraded performance (-0.27% slowdown) because speculative data prefetches displaced critical instructions in L2C. However, when paired with prefetch-aware Dyn-IRRIP, the co-design delivers **+1.69% speedup** and achieves the **lowest L2C load MPKI across all experiments (23.06)** while retaining 15,391 useful prefetches. On **Trace 3**, the co-design cuts L1D misses by **15.38%** and L2C MPKI from 3.95 down to **3.18**, proving strong architectural synergy across workload types.

6. Conclusion

This assignment demonstrates that memory hierarchy optimizations cannot operate in isolation. While data prefetching (SPP) effectively tackles regular data streams (achieving 15.33% coverage on Trace 3), it risks polluting shared cache tiers. Instruction-conscious replacement (I-RRIP) successfully shields the serial frontend bottleneck (slashing Trace 2 instruction misses by 24.5%). Finally, their prefetch-aware co-design harmonizes both mechanisms, achieving superior overall IPC and the lowest overall cache miss rates across diverse execution traces.